USE CASE DOCUMENT

AmuseFlow System

Development of an Online Theme Park Attraction Reservation System for Glorious Fantasyland

Project Title

AmuseFlow System: Development of an Online Theme Park Attraction Reservation System for Glorious Fantasyland

System Description

AmuseFlow System is a web-based theme park attraction reservation and management platform designed for Glorious Fantasyland. The system allows visitors to register, browse available attractions, view schedules, and reserve attraction slots online. It also provides administrators with tools to manage attractions, attraction schedules, bookings, user accounts, and activity logs. The system aims to reduce long queues, improve booking efficiency, and provide better monitoring of attraction reservations and system transactions.

Actors

Actor	Description
Visitor (User)	Registers, logs in, browses attractions, books schedules, views and cancels bookings, and optionally reviews completed bookings.
Admin (Park Manager)	Manages attractions, schedules, bookings, users, dashboards, passwords, attraction bundles, notifications, system settings, and activity logs — sees every admin's activity, not filtered to their own.
Ride Attendant	Views assigned schedules, verifies visitors, collects payment, and marks attractions as completed.

Use Case List

Use Case ID	Use Case Name	Primary Actor
UC-01	Register Account	Visitor
UC-02	Login	Visitor/Admin/Attendant
UC-03	Logout	Visitor/Admin/Attendant
UC-04	View User Dashboard	Visitor
UC-05	Browse Attractions	Visitor
UC-06	View Attraction Details	Visitor
UC-07	Book Attraction Schedule	Visitor
UC-08	View Booking History	Visitor
UC-09	Cancel Booking	Visitor
UC-10	Manage Attractions	Admin
UC-11	Restore Attraction	Admin
UC-12	Manage Attraction Schedules	Admin
UC-13	Manage Bookings	Admin
UC-14	Approve / Reject Booking	Admin
UC-15	View Dashboard Analytics	Admin
UC-16	Manage Users	Admin
UC-17	View Activity Logs	Admin
UC-18	Reset User Password	Admin
UC-19	Verify Visitor and Complete Attraction	Ride Attendant
UC-20	View Assigned Attraction Schedule	Ride Attendant
UC-21	View My Activity Log	Visitor/Ride Attendant
UC-22	Change My Password / Contact Number	Visitor/Admin/Attendant
UC-23	Change User Role	Admin
UC-24	Create Staff Account	Admin
UC-25	Activate / Deactivate User Account	Admin
UC-26	Auto-Complete Expired Schedules	System (Admin-triggered)
UC-27	Auto-Flag Missed Bookings	System (Admin-triggered)
UC-28	Create Attraction Bundle	Admin
UC-29	Update Attraction Bundle	Admin
UC-30	Delete / Restore Attraction Bundle	Admin
UC-31	Book Attraction Bundle	Visitor
UC-32	Auto-Complete Finished Bundles	System (Admin-triggered)

UC-33	Receive Automatic Notifications	System (triggered by other use cases)
UC-34	View & Manage My Notifications	Visitor/Ride Attendant
UC-35	View & Manage System-Wide Notifications	Admin
UC-36	Leave an Attraction Review	Visitor
UC-37	Configure Attraction Validation Settings	Admin
UC-38	Generate Attraction & Bundle Rating Reports	Admin
UC-39	Configure Rider Categories	Admin

Use Case Details

UC-01 - Register Account

Primary Actor: Visitor

Description: Visitor performs the register account process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-02 - Login

Primary Actor: All Actors (Visitor, Admin, Ride Attendant)

Description: Any registered user logs into the system with their username and password to receive an authenticated session.

Preconditions: The user has a registered, active account and knows their username and password.

Main Flow:

1. Actor submits username and password on the login screen.
2. System retrieves the account by username and verifies it exists and is active (not deactivated).
3. System verifies the submitted password against the stored password hash.
4. System generates a session token containing the user's ID, username, and role, and returns it along with the user's profile information.
5. System records the login as an Activity Log entry under the User module, action "Login," tagged with the actor’s actual role — every login is now logged, not just Admin logins as before.
6. Actor is redirected to the appropriate portal (Admin, Ride Attendant, or Visitor) based on their role.

Alternative Flow:

- If the username does not exist or the password does not match, the system returns an "Invalid username or password" error and records no log entry.
- If the account has been deactivated, the system returns an "Account is deactivated. Please contact the administrator." error.

Postconditions: A valid session is issued to the actor, and a Login entry is recorded in the Activity Logs carrying the actor's role, regardless of whether the actor is an Admin, Ride Attendant, or Visitor.

UC-03 - Logout

Primary Actor: All Actors (Visitor, Admin, Ride Attendant)

Description: An authenticated user ends their current session.

Preconditions: The actor is currently logged in with a valid session.

Main Flow:

1. Actor selects "Sign out" and confirms via the logout confirmation dialog.
2. System reads the actor's user ID and role from their current session.

3. System records the logout as an Activity Log entry under the User module, action "Logout," tagged with the actor's role — every logout is now logged for all three roles, not just Admin as before.
4. System returns a success response.
5. Client discards the session token and redirects the actor to the login screen.

Alternative Flow:

- If the logout request fails due to a network error, the client still discards the local session and redirects to the login screen so the actor is never left stuck.

Postconditions: The actor's session ends, and a Logout entry is recorded in the Activity Logs attributing the action to the correct role.

UC-04 - View User Dashboard

Primary Actor: Visitor

Description: Visitor performs the view user dashboard process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-05 - Browse Attractions

Primary Actor: Visitor

Description: Visitor performs the browse attractions process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-06 - View Attraction Details

Primary Actor: Visitor

Description: Visitor performs the view attraction details process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.

4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-07 - Book Attraction Schedule

Primary Actor: Visitor

Description: A Visitor reserves a slot on an available attraction schedule.

Preconditions: Visitor is logged in, and the target attraction schedule has at least one available slot and is not Cancelled or Completed.

Main Flow:

1. Visitor selects an attraction and an available schedule slot.
2. System verifies the schedule exists, has available slots, and its status is not Full, Cancelled, or Completed.
3. System retrieves the attraction's price.
4. System validates the guest's information (birthdate/age, height, and weight) against the attraction's optional MinHeightCm/MaxHeightCm, MinAgeYears/MaxAgeYears, and MinWeightKg/MaxWeightKg restrictions, where configured — a guest who fails any configured restriction cannot proceed.
5. System creates a new booking with a generated booking code, status "Pending," and payment status "Unpaid."
6. System decrements the schedule's available slot count.
7. System records the booking as an Activity Log entry under the Booking module, action "Book Attraction," tagged with the Visitor's role — this step was never logged before.
8. System displays a success message asking the Visitor to pay to confirm the attraction.

Alternative Flow:

- If the schedule has no available slots, or its status is Cancelled or Completed, the system rejects the booking with an explanatory message and creates no booking or log entry.
- If the guest does not meet one or more of the attraction's configured height, age, or weight restrictions, the system rejects the booking with a message identifying which restriction failed, and creates no booking or log entry.

Postconditions: A new Pending booking exists, the schedule's available slot count is reduced by one, and a "Book Attraction" entry is recorded in the Activity Logs under the Visitor role.

UC-08 - View Booking History

Primary Actor: Visitor

Description: Visitor performs the view booking history process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-09 - Cancel Booking

Primary Actor: Visitor

Description: A Visitor cancels one of their own bookings.

Preconditions: Visitor is logged in and owns the booking being cancelled; the booking is not already Completed or Cancelled.

Main Flow:

1. Visitor selects a booking from their booking history and requests cancellation.
2. System verifies the booking exists and belongs to the requesting Visitor.
3. System verifies the booking's status is not already Completed or Cancelled.
4. System updates the booking's status to "Cancelled."
5. System increments the schedule's available slot count, freeing the slot for other visitors.
6. System records the cancellation as an Activity Log entry under the Booking module, action "Cancel Booking," tagged with the Visitor's role — this step was never logged before.

Alternative Flow:

- If the booking belongs to a different visitor, the system returns an "Unauthorized" error.
- If the booking is already Completed or Cancelled, the system rejects the request with an explanatory message.

Postconditions: The booking's status is set to "Cancelled," its schedule slot is released, and a "Cancel Booking" entry is recorded in the Activity Logs under the Visitor role.

UC-10 - Manage Attractions

Primary Actor: Admin

Description: Admin performs the manage attractions process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-11 - Restore Attraction

Primary Actor: Admin

Description: Admin performs the restore attraction process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-12 - Manage Attraction Schedules

Primary Actor: Admin

Description: Admin performs the managed attraction schedules process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-13 - Manage Bookings

Primary Actor: Admin

Description: Admin performs the manage bookings process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-14 - Approve/Reject Booking

Primary Actor: Admin

Description: Admin performs the approve/reject booking process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-15 - View Dashboard Analytics

Primary Actor: Admin

Description: Admin performs the view dashboard analytics process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-16 - Manage Users

Primary Actor: Admin

Description: Admin performs the managed users process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-17 - View Activity Logs

Primary Actor: Admin

Description: An Admin reviews the history of actions performed across the system. The page defaults to Admin-performed actions only, and is not restricted to the currently logged-in admin — every admin's actions are visible.

Preconditions: Admin is logged in.

Main Flow:

1. Admin opens the Activity Logs page.
2. System retrieves activity log entries filtered to the Admin role by default, spanning every admin account — never limited to just the logged-in admin's own actions.
3. Admin may narrow the results using the search box (matches action, module, details, or username), the module filter (Attraction, Schedule, Booking, User), or the date range picker.
4. System groups the matching entries by date and displays them as a timeline, each entry showing its module badge, role badge, action, short details, performer, and relative time.
5. Admin clicks any entry to open its detail view, showing who performed the action, the exact timestamp, the module, and the full details text.

Alternative Flow:

- If no entries match the current filters, the system displays an empty state instead of a list.

Postconditions: The Admin has visibility into every admin's actions system-wide. Ride Attendant and Visitor activity is not shown on this page — each of those roles instead sees only their own activity through their own portal's Activity Logs panel.

UC-18 - Reset User Password

Primary Actor: Admin

Description: Admin performs the reset user password process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.
3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-19 - Verify Visitor and Complete Attraction

Primary Actor: Ride Attendant

Description: A Ride Attendant checks a visitor into their scheduled attraction, collects payment, and marks the attraction as completed. This is two distinct steps in the system: collecting payment, then completing the attraction.

Preconditions: Ride Attendant is logged in; the booking exists, is Approved, and falls within its schedule's check-in window (from 15 minutes before the scheduled start time through the scheduled end time).

Main Flow:

1. Ride Attendant looks up a visitor's booking by its booking code.
2. System verifies the booking is Approved, not already paid, and within the schedule's check-in window.
3. Ride Attendant collects the attraction payment; system records the payment amount against the booking and records an Activity Log entry under the Booking module, action "Collect Payment," tagged with the Ride Attendant’s role — previously never logged.
4. Once payment is collected, the Ride Attendant marks the visitor as having completed the attraction; system verifies the booking is Approved and paid, updates its status to "Completed," and records a second Activity Log entry, action "Complete Attraction," also tagged with the Ride Attendant’s role — likewise previously never logged.

Alternative Flow:

- If the check-in window hasn't opened yet or has already closed, the system rejects the action and states when check-in opens or that it has closed.
- If payment hasn't been collected yet, the system blocks attraction completion and instructs the attendant to collect payment first.
- If the booking is Cancelled or Rejected, the system blocks payment collection entirely.

Postconditions: The booking’s payment is recorded and its status becomes "Completed." Two Activity Log entries — "Collect Payment" and "Complete Attraction" — are recorded under the Ride Attendant’s role, making Ride Attendant activity fully visible when filtering the Activity Logs by role.

UC-20 - View Assigned Attraction Schedule

Primary Actor: Ride Attendant

Description: Ride Attendant performs the view assigned attraction schedule process.

Preconditions: User has appropriate permissions and system is available.

Main Flow:

1. Actor starts the process.
2. System validates the request.

3. System processes the action.
4. System saves changes.
5. Success message is displayed.

Alternative Flow: If validation fails, the system displays an appropriate error message.

Postconditions: Requested action is completed and recorded in the Activity Logs.

UC-21 - View My Activity Log

Primary Actor: Visitor / Ride Attendant

Description: A Visitor or Ride Attendant opens their portal header's "Activity logs" panel to review their own recent actions — a self-service view that did not exist before this session (Ride Attendant activity was previously a hardcoded placeholder list, and Visitor activity was only inferred from booking statuses, not real log entries).

Preconditions: Actor is logged in as a Visitor or Ride Attendant.

Main Flow:

1. Actor opens "Activity logs" from their portal's account menu.
2. System calls the same activity log endpoint the Admin page uses, but automatically forces the request to the actor's own user ID and clears any role filter — the actor can never see anyone else's activity.
3. Actor may narrow the results using the search box, module filter, or date range, the same controls available on the Admin page.
4. System groups the actor's own entries by date and displays them as a timeline.
5. Actor clicks an entry to view its full detail (module, timestamp, performer, and details).

Alternative Flow:

- If the actor has no logged activity yet, the system displays an empty state instead of a list.

Postconditions: The actor sees only their own historical actions (bookings, cancellations, payments collected, attraction completions, logins/logouts) — never another user's, even though the request goes through the same endpoint the Admin page uses.

UC-22 - Change My Password / Contact Number

Primary Actor: Visitor / Admin / Ride Attendant (self-service)

Description: Any logged-in actor updates their own password and/or contact number from their account menu, distinct from an Admin resetting someone else's password (UC-18).

Preconditions: Actor is logged in.

Main Flow:

1. Actor opens "Change password" from their account menu.
2. System fetches the actor's current contact number and locks that field by default so it isn't accidentally changed.
3. Actor optionally unlocks the contact number to edit it, and/or enters a new password (8+ characters, an uppercase letter, a number, and a special character) with confirmation.
4. System validates that any new password meets all requirements and matches its confirmation, and that any edited contact number is a valid PH mobile number.
5. System updates the password hash and/or contact number, recognizing this as the actor changing their own credentials rather than an admin changing someone else's.
6. System records the change as an Activity Log entry under the User module, with detail text noting it was a self-service change made by the account owner.

Alternative Flow:

- If nothing was actually changed, the system rejects the save with "No changes to save."
- If the new password doesn't meet all requirements, or the two password fields don't match, the system shows an error and does not save.

Postconditions: The actor's password and/or contact number is updated, and an Activity Log entry records the self-service change, distinguishing it from an admin-driven reset.

UC-23 - Change User Role

Primary Actor: Admin

Description: An Admin changes another user's role among Visitor, Admin, or Ride Attendant.

Preconditions: Admin is logged in; the target user exists.

Main Flow:

1. Admin selects a user and requests a role change to one of Visitor, Admin, or Ride Attendant.
2. System validates that the requested role is one of the three allowed roles.
3. System verifies the target user exists.
4. System updates the user's role and records the change as an Activity Log entry under the User module, action "Change Role," noting the old and new role.

Alternative Flow:

- If the requested role isn't one of the three valid roles, or the target user doesn't exist, the system rejects the change with an explanatory error.

Postconditions: The user's role is updated, and an Activity Log entry records who changed it and the before/after roles.

UC-24 - Create Staff Account

Primary Actor: Admin

Description: An Admin creates a new Admin or Ride Attendant staff account (Visitor accounts are created only through self-registration, UC-01).

Preconditions: Admin is logged in; the desired username is not already taken.

Main Flow:

1. Admin submits a new staff account's details, including a role of either Admin or Ride Attendant.
2. System validates the role is one of the two staff roles.
3. System verifies the username isn't already taken.
4. System hashes the provided password and creates the account.
5. System records the creation as an Activity Log entry under the User module, action "Create Staff," noting the new account's role and username.

Alternative Flow:

- If the role isn't Admin or Ride Attendant, or the username is already taken, the system rejects the request with an explanatory error.

Postconditions: A new staff account exists with the requested role, and an Activity Log entry records its creation.

UC-25 - Activate / Deactivate User Account

Primary Actor: Admin

Description: An Admin toggles whether a user account is active, controlling whether that user is allowed to log in.

Preconditions: Admin is logged in; the target user exists.

Main Flow:

1. Admin selects a user and toggles their account between active and inactive.
2. System verifies the target user exists.
3. System updates the account's active flag.

4. System records the change as an Activity Log entry under the User module, action "Toggle User Active," noting the new status.

Alternative Flow:

- If the target user doesn't exist, the system rejects the request.
- A deactivated user who attempts to log in receives "Account is deactivated. Please contact the administrator." (see UC-02).

Postconditions: The user's account is activated or deactivated, an Activity Log entry records the change, and a deactivated account can no longer log in until reactivated.

UC-26 - Auto-Complete Expired Schedules

Primary Actor: System (Admin-triggered maintenance)

Description: The system automatically marks attraction schedules as Completed once their scheduled end time has passed.

Preconditions: Admin triggers the maintenance action.

Main Flow:

1. Admin triggers the auto-complete maintenance action.
2. System finds all schedules whose end time has passed and are not already Completed or Cancelled.
3. System updates each qualifying schedule's status to "Completed."
4. If one or more schedules were updated, system records a single Activity Log entry under the Schedule module, action "Auto-Complete Schedules," noting how many schedules were affected.

Alternative Flow:

- If no schedules qualify, the system completes with a count of zero and writes no log entry.

Postconditions: All expired, still-open schedules are marked Completed, and an Activity Log entry records the batch update whenever at least one schedule was affected.

UC-27 - Auto-Flag Missed Bookings

Primary Actor: System (Admin-triggered maintenance)

Description: The system automatically flags unpaid Approved bookings as "Missed" once their schedule's start time has passed.

Preconditions: Admin triggers the maintenance action.

Main Flow:

1. Admin triggers the auto-flag maintenance action.
2. System finds all Approved bookings that remain unpaid after their schedule's start time has passed.
3. System updates each qualifying booking's status to "Missed."
4. If one or more bookings were flagged, system records a single Activity Log entry under the Booking module, action "Auto-Flag Missed Bookings," noting how many bookings were affected.

Alternative Flow:

- If no bookings qualify, the system completes with a count of zero and writes no log entry.

Postconditions: Unpaid, expired-window Approved bookings are marked Missed, and an Activity Log entry records the batch update whenever at least one booking was affected.

UC-28 - Create Attraction Bundle

Primary Actor: Admin

Description: An Admin bundles two or more attractions into a single discounted "Attraction Bundle" package for one specific date, locking each included attraction to one of its Bundle-type schedules on that date.

Preconditions: Admin is logged in; at least two attractions exist, each with an available Bundle-type schedule on the intended bundle date.

Main Flow:

1. Admin opens "Create Bundle" and enters the bundle's name, description, price, image, and a single bundle date.
2. Admin selects at least two attractions to bundle, and for each attraction picks one of its schedules.
3. System verifies at least two distinct attractions were selected, with no attraction repeated, and that every selected attraction has exactly one matching schedule.
4. System verifies each (attraction, schedule) pair is real, belongs to that attraction, is not Cancelled or Completed, is tagged Type = Bundle (never a Regular schedule), and falls on the same date as the bundle.
5. System verifies none of the bundled attractions' locked schedules overlap in time with each other, since a visitor can't be at two attractions at once.
6. System creates the bundle, computing its Status as "Active" (or "Completed" if the chosen date is already in the past), and links each selected attraction/schedule pair.
7. System records the creation as an Activity Log entry under the Attraction module, action "Create Attraction Bundle," noting the bundled attraction count.

Alternative Flow:

- If fewer than two distinct attractions are selected, any attraction repeats, a schedule doesn't belong to its attraction, is Cancelled/Completed, isn't a Bundle-type schedule, is on a different date than the bundle, or two schedules overlap in time, the system rejects the request with a specific explanatory message and creates nothing.

Postconditions: A new Attraction Bundle exists with its bundled attractions and locked schedules, and an Activity Log entry records its creation.

UC-29 - Update Attraction Bundle

Primary Actor: Admin

Description: An Admin edits an existing Attraction Bundle's details or its bundled attractions/schedules, as long as the bundle hasn't already started.

Preconditions: Admin is logged in; the bundle exists; the earliest call time among its bundled attractions has not yet passed.

Main Flow:

1. Admin opens an existing bundle and edits its name, description, price, image, date, and/or its bundled attractions/schedules.
2. System verifies the bundle exists and that the earliest call time among its currently bundled attractions has not yet passed.
3. System re-runs the same bundling validation used at creation (at least two distinct attractions, matching Bundle-type schedules on the same date, no time overlaps).
4. System applies the changes, recomputing the bundle's Status based on its (possibly new) date.
5. System replaces the bundle's attraction/schedule links with the newly submitted set.
6. System records the update as an Activity Log entry under the Attraction module, action "Update Attraction Bundle."

Alternative Flow:

- If the earliest call time among the bundle's bundled attractions has already passed, the system rejects the edit as locked.
- Any bundling validation failure (as in UC-28) rejects the update with an explanatory message and leaves the bundle unchanged.

Postconditions: The bundle's details and/or bundled attractions are updated (unless locked or rejected), and an Activity Log entry records the change.

UC-30 - Delete / Restore Attraction Bundle

Primary Actor: Admin

Description: An Admin soft-deletes an Attraction Bundle that's no longer offered, or restores a previously deleted one.

Preconditions: Admin is logged in; for deletion, the bundle exists and its earliest call time hasn't passed, and either none of its bundled schedules are Cancelled or it has no active (Pending/Approved) reservations.

Main Flow:

1. Admin requests to delete a bundle.
2. System verifies the bundle's earliest call time hasn't passed; if it has, the deletion is blocked.
3. System checks whether every bundled attraction's schedule is Cancelled; if not, system counts active (Pending/Approved) bookings against the bundle.
4. If any active bookings exist and not all schedules are cancelled, system rejects the deletion, reporting how many reservations still need to be resolved.
5. Otherwise, system marks the bundle as deleted and records an Activity Log entry, action "Delete Attraction Bundle."
6. To reverse this, Admin selects the deleted bundle and restores it; system un-marks it as deleted and records an Activity Log entry, action "Restore Attraction Bundle."

Alternative Flow:

- If the bundle is already locked (call time passed) or has unresolved active reservations, deletion is rejected with an explanatory message.
- If the bundle doesn't exist, the request is rejected.

Postconditions: The bundle is hidden from active listings (soft-deleted) or made visible again (restored), with an Activity Log entry recording whichever action occurred.

UC-31 - Book Attraction Bundle

Primary Actor: Visitor

Description: A Visitor reserves an entire Attraction Bundle bundle in a single booking, covering every included attraction's locked schedule at once.

Preconditions: Visitor is logged in; the bundle is Active (not Completed or deleted); every bundled attraction's locked schedule still has an available slot.

Main Flow:

1. Visitor selects an Active bundle to book.
2. System verifies the bundle exists, is not deleted, and is still Active.
3. System verifies every one of the bundle's bundled schedules still has at least one available slot and isn't Cancelled or Completed.
4. System validates the guest's information (birthdate/age, height, and weight) against EVERY bundled attraction's own optional height, age, and weight restrictions — the guest must qualify for all of them, not just one, wherever a bundled attraction has any configured.
5. System creates a single booking (linked to the bundle, with no single schedule of its own) with a generated booking code, status "Pending," payment status "Unpaid," and the bundle's price as the payment amount.
6. System records each bundled schedule against the booking and decrements each one's available slot count.
7. System records the reservation as an Activity Log entry under the Booking module, action "Book Attraction Bundle."

Alternative Flow:

- If the bundle is deleted, not Active, or any one of its bundled schedules has no available slots or is Cancelled/Completed, the system rejects the booking and creates nothing.
- If the guest fails a height, age, or weight restriction on any one of the bundled attractions, the system rejects the booking with a message identifying which attraction and restriction failed, and creates nothing.

Postconditions: A single Pending, Unpaid booking exists covering every attraction in the bundle, each bundled schedule's slot count is reduced by one, and an Activity Log entry records the reservation.

UC-32 - Auto-Complete Finished Bundles

Primary Actor: System (Admin-triggered maintenance)

Description: The system automatically flips Active Attraction Bundles to Completed once their bundle date has passed, mirroring the auto-complete jobs for schedules and bookings.

Preconditions: Admin triggers the maintenance action (the same scheduled background job that runs the schedule and booking auto-updates).

Main Flow:

1. The maintenance job runs on its schedule.
2. System finds all bundles that are Active, not deleted, and whose bundle date is earlier than the current date.
3. System updates each qualifying bundle’s status to "Completed."

Alternative Flow:

- If no bundles qualify, the system completes with no changes.

Postconditions: Every Active bundle whose date has passed is marked Completed, keeping its status consistent with the attractions/schedules it bundles.

UC-33 - Receive Automatic Notifications

Primary Actor: System (triggered by other use cases)

Description: The system automatically creates an in-app notification for a Visitor or Ride Attendant whenever an action elsewhere in the system affects them directly — being assigned to a schedule, or having a booking approved, rejected, cancelled, or paid, or being affected by a schedule cancellation or reopening.

Preconditions: The triggering action (schedule assignment, booking status change, payment collection, schedule cancel/reopen) has just occurred.

Main Flow:

1. An Admin assigns a Ride Attendant to a schedule (a new assignment or a swap to a different attendant); system creates a "New schedule assigned" notification for that attendant.
2. An Admin approves or rejects a booking; system creates a "Booking approved" or "Booking rejected" notification for the visitor who made it.
3. A Visitor cancels their own booking; system creates a "Booking cancelled" notification for that same visitor, confirming the cancellation.
4. A Ride Attendant collects payment for a booking; system creates a "Payment confirmed" notification for the visitor.
5. An Admin cancels or reopens a schedule; system creates a "Schedule cancelled"/"Schedule reopened" notification for the assigned attendant, and a "Attraction schedule cancelled"/"Attraction schedule reopened" notification for every visitor with a Pending or Approved booking against that schedule, including bundle bookings that include it.
6. Each notification is stored against the recipient's user ID, tagged with its module and an optional related record ID, and starts unread.

Alternative Flow:

- Reassigning a schedule to the same attendant it already had does not generate a duplicate notification.
- A schedule edit that doesn't change its attendant, or a booking action with no status change, creates no notification.

Postconditions: The affected Visitor or Ride Attendant has a new unread notification describing exactly what changed and why.

UC-34 - View & Manage My Notifications

Primary Actor: Visitor / Ride Attendant

Description: A Visitor or Ride Attendant checks their notification bell to see what's happened to their bookings or schedule assignments, and clears the unread badge as they read them.

Preconditions: Actor is logged in.

Main Flow:

1. Actor sees an unread-count badge on their portal's notification bell if they have unread notifications.
2. Actor opens the bell to view their notifications, newest first, each showing its title, message, and relative time.
3. System scopes the list strictly to the actor's own user ID — the actor never sees another user's notifications.
4. Actor clicks "Mark all as read" (or opening the panel triggers it), clearing the unread badge.

Alternative Flow:

- If the actor has no notifications yet, the bell shows no badge and the panel displays an empty state.

Postconditions: The actor is aware of recent changes affecting their bookings or assignments, and their unread count reflects what they've actually seen.

UC-35 - View & Manage System-Wide Notifications

Primary Actor: Admin

Description: An Admin reviews notifications sent to any Visitor or Ride Attendant across the whole system, defaulting to visitor booking-cancellation notifications so the page opens focused on the cancellations an Admin actually needs to react to.

Preconditions: Admin is logged in.

Main Flow:

1. Admin opens the Notifications page.
2. System retrieves notifications system-wide, scoped by default to the "Booking cancelled" title (a visitor cancelling their own booking) so the page isn’t cluttered with every schedule-assignment or payment notification.
3. Admin may narrow results with a date range filter.
4. System displays each notification with its recipient's name, username, and role alongside the notification's own title, message, and time.
5. Admin clicks a notification (or "Mark all as read") to mark it read; system updates it without regard to whose notification it is, unlike the self-service Visitor/Attendant view.
6. The Notifications nav item shows an unread-count badge scoped to the same "Booking cancelled" default, matching what the page itself displays.

Alternative Flow:

- If no notifications match the current date filter, the system displays an empty state.

Postconditions: The Admin has visibility into system-wide notification activity (cancellations by default), and can mark entries read individually or in bulk.

UC-36 - Leave an Attraction Review

Primary Actor: Visitor

Description: A Visitor optionally rates (1–5 stars) and comments on a booking after it's completed and paid — either a regular attraction booking or an entire Attraction Bundle bundle.

Preconditions: Visitor is logged in and owns the booking; the booking's status is Completed and its payment status is Paid; the booking has not already been reviewed.

Main Flow:

1. Visitor opens a Completed and Paid booking from their booking history and selects "Leave a review" (clearly marked optional).
2. System verifies the booking exists, belongs to the requesting Visitor, is Completed and Paid, and has not already been reviewed.
3. Visitor submits a star rating and an optional text comment.
4. If the booking is for a regular attraction, system attaches the review to that attraction (via the booking's schedule), so it contributes to that attraction's own average rating.
5. If the booking is for an Attraction Bundle, system creates the review with no single attraction attached — one review represents the entire bundle and contributes only to the bundle's own separate average rating, never to any individual bundled attraction's rating.
6. System records the review as an Activity Log entry under the Review module, action "Leave Review," noting the rating and the name of the reviewed attraction or bundle.
7. System displays a thank-you message.

Alternative Flow:

- If the booking isn't Completed and Paid yet, already has a review, or doesn't belong to the requesting visitor, the system rejects the request with an explanatory error and creates no review.

Postconditions: A new optional review exists tied to the booking (attached to its attraction, or left unattached for a bundle bundle), contributing to the appropriate average rating display. A "Leave Review" entry is recorded in the Activity Logs under the Visitor role. Leaving a review is always optional — a booking's completion never requires one.

UC-37 - Configure Attraction Validation Settings

Primary Actor: Admin

Description: An Admin configures the floor/ceiling bounds that constrain what values can be entered for an attraction's height, age, and weight restriction fields, replacing what used to be fixed limits baked into the code.

Preconditions: Admin is logged in.

Main Flow:

1. Admin opens the Settings page and expands the “Attraction Validations” section.
2. System displays the current allowed floor and ceiling for each of the six attraction restriction fields: minimum and maximum height, age, and weight.
3. Admin adjusts one or more floor/ceiling values and saves the changes.
4. System verifies each pair's ceiling is strictly greater than its floor, then persists the new bounds as a single settings record.
5. System records the update as an Activity Log entry under the Settings module, action “Update Attraction Validation Settings.”
6. From then on, creating or updating an attraction validates its restriction fields against these current bounds instead of a fixed range — widening or narrowing the bounds here immediately changes what the attraction form accepts, with no code change or redeploy required.

Alternative Flow:

- If a submitted floor/ceiling pair doesn't have the ceiling strictly greater than the floor, the system rejects the update with an explanatory error and leaves the previous bounds in place.

Postconditions: The stored attraction-validation bounds reflect the Admin's changes, and every subsequent attraction create/update is validated against them.

UC-38 - Generate Attraction & Bundle Rating Reports

Primary Actor: Admin

Description: An Admin reviews average visitor ratings by month, attraction, and Attraction Bundle over a chosen date range, and can print the results as a letterhead report.

Preconditions: Admin is logged in.

Main Flow:

1. Admin opens the Reports page, which defaults to the current calendar month.

2. Admin optionally narrows the report with a scope filter (all attractions and bundles, only Attractions, or only Attraction Bundles), an interactive search combobox to pick one specific attraction or bundle, and/or a custom date range picker.

3. System computes the monthly average rating trend for the selected scope and period — zero-filling any month with no reviews so the chart always spans the full selected range — along with an overall average rating and total review count.

4. System also computes a per-attraction/per-bundle rating breakdown for the period, leaving out any attraction or bundle that received no reviews during that period; the remaining results feed a breakdown table the Admin can sort by name or rating, ascending or descending, and page through.

5. Admin clicks “Print report” to generate a letterhead version of the report — the park's logo and address, the applied scope and period, the summary figures (overall average rating, total reviews, and how many attractions/bundles were rated), the trend chart with its exact monthly figures, the full breakdown table, and a signature block with the preparer's name and a blank line reserved for a verifier's signature.

Alternative Flow:

- If no reviews exist for the selected scope and period, the trend chart shows a flat line at zero for every month and the breakdown table is empty, with “Entities Rated” reading 0.

Postconditions: The Admin has an accurate, period-scoped view of visitor satisfaction across attractions and Attraction Bundles, optionally saved as a signed, printable record.

UC-39 - Configure Rider Categories

Primary Actor: Admin

Description: An Admin defines Kid, Teen, and Adult age, height, and weight ranges used to tag which attractions and attraction bundles each category applies to, and to tighten a tagged attraction's own restriction fields to that category's range.

Preconditions: Admin is logged in.

Main Flow:

1. Admin opens the Settings page and expands the “Rider Categories” section, which lists Kid, Teen, and Adult, each with its own age, height, and weight range, plus a read-only “General Admission” card that always mirrors the widest range the Attraction Validation Settings above allow.

2. Admin adjusts one or more ranges for a category and saves; the system rejects the change if a category's age range would overlap a sibling's (Kid/Teen/Adult are mutually exclusive by age), if any value falls outside the current Attraction Validation Settings bounds, or if a minimum isn't strictly less than its maximum — height and weight are allowed to overlap between categories.

3. When creating or editing an attraction or attraction bundle, Admin optionally tags it with one or more rider categories; selecting a category tightens that attraction's own Min/Max age, height, and weight fields to stay within the selected category's range — or the combined range, if more than one category is selected — instead of the broader Attraction Validation Settings bounds.

4. System records the category tag and displays it as a badge on the attraction to Admins, Visitors, and Ride Attendants, so everyone can see at a glance who the attraction is intended for.

5. If the Admin later tightens the Attraction Validation Settings — the floor/ceiling bounds above — such that an existing category's saved range falls outside the new bounds, the system automatically adjusts that category's value back inside the new range the moment the settings are saved, and records the adjustment in the Activity Logs.

Alternative Flow:

- If saving a category would push its age range into a sibling's, the system rejects it with the sibling's current range shown in the error message.

- If a settings change is severe enough that clamping a category's minimum would reach or pass its own maximum, that pair is left as-is instead of being forced invalid; the next manual edit to that category will surface the conflict.

Postconditions: The three rider categories stay within the Attraction Validation Settings bounds at all times, either through the Admin's own edits or an automatic adjustment, and every attraction/bundle tagged with a category enforces that category's tightened range.